

DSAIT4335 Recommender Systems

RecBole instruction

In this document, you will learn how to work with RecBole, a recommendation tool for running experiments. The first necessary step is learning how to (a) setup this tool, and (b) how to run the experiment given a dataset and a recommendation model.

1) Installation

You can clone the repository and install it as follows:

```
git clone https://github.com/masoudmansoury/RecBole_DSAIT4335.git
cd RecBole_DSAIT4335
pip install -e . --verbose
```

Verify the installation:

```
python -c "import recbole; print(recbole.__version__)"
```

2) Dataset

Dataset already provided: `dataset/ml-100k`, in RecBole's atomic-file format (`.inter`, `.item`, optionally `.user`). Column types are declared in the file header, e.g.:

```
user_id:token  item_id:token  rating:float  timestamp:float
196      242      3      881250949
```

* To bring your own dataset, drop `<name>.inter` (and `.item/.user` if you have them) into `dataset/<name>/` following the same `field:type` header convention.

You do **not** need to run a separate split script to *train* a model – `run_recbole.py` performs the split internally, deterministically, from `eval_args` + `seed` in your config.

A dedicated split-export script is only needed for part 4.

3) Train and tune models

3.1) Configuration files

Each model/dataset pair gets a YAML file, e.g. `recbole/config/BPR/ml-100k.yaml`:

```

model: BPR
dataset: ml-100k
data_path: dataset/

embedding_size: 64
train_neg_sample_args:
  distribution: uniform
  sample_num: 1

epochs: 20
train_batch_size: 2048
learner: adam
learning_rate: 0.001
eval_step: 10
stopping_step: 20

eval_args:
  split:
    RS: [0.8, 0.1, 0.1]      # train / valid / test ratio
  group_by: user
  order: R0                  # random order split
  mode: full                 # full-ranking (all items) evaluation
metrics: [Recall, MRR, NDCG, Hit, Precision]
topk: [10]
valid_metric: MRR@10

```

Global defaults (that apply unless overridden in your YAML) live in

`recbole/properties/overall.yaml` – notably `seed: 2020` and `reproducibility: True`.

Keep `seed` and `eval_args` identical across every model you plan to compare.

3.2) Single training run

Via `run_recbole.py`:

```

python run_recbole.py \
  --model <model_name> --dataset <dataset_name> \
  --config_files recbole/config/<model_name>/<dataset_name>.yaml

```

- For dataset in this assignment, you can set it ml-100k.
- The models are:
 - `BPR`: pairwise ranking optimization based on matrix factorization
 - `FISM`: low-rank neighborhood model

- **UserKNN**: user-based collaborative filtering
- **ItemKNN**: item-based collaborative filtering
- **SLIMElastic**: Sparse Linear Method
- **EASE**: Embarrassingly Shallow AutoEncoder
- **Pop**: a simple model recommending most popular items
- **Random**: a simple model recommending random items

Example for BPR on ml-100k (implementation of BPR in RecBole is based on Matrix Factorization):

```
python run_recbole.py \
  --model BPR --dataset ml-100k \
  --config_files recbole/config/BPR/ml-100k.yaml
```

* An exception about item-based and user-based CF methods: you can run them as follows:

Item-based CF

```
python run_recbole.py \
  --model ItemKNN --dataset ml-100k \
  --config_files recbole/config/ItemKNN/ml-100k.yaml
```

User-based CF:

```
python run_recbole.py \
  --model ItemKNN --dataset ml-100k \
  --config_files recbole/config/UserKNN/ml-100k.yaml
```

On completion the model checkpoint is written to `saved/<Model>-<timestamp>.pth`. It contains `config`, `epoch`, `state_dict`, `other_parameter`, and `optimizer` – **not** the split or the recommendation lists (that's part 4).

4) Save the split data and the recommendation results

Why: the model checkpoint alone does not give you a portable, human-readable record of:

- (a) which interactions ended up in train/valid/test,
- or (b) what each model actually recommended.

Both are needed if you want to inspect results outside RecBole, put them in a report, or recompute a metric later without retraining.

RecBole *does* have a built-in (pickle-based) mechanism for caching splits – `save_dataset: True` / `save_data_loaders: True` in your config, handled by `recbole/data/utils.py`.

This is useful for speeding up repeated runs but the pickles are RecBole-internal objects, not something you can open in a spreadsheet or read from another tool. For this assignment, you will add two small scripts that export **plain TSV/JSON with the original dataset's user/item IDs** (not RecBole's internal integer IDs). It can help you to compare the results with the ground truth, and also match with the metadata of items/users for further beyond-accuracy analysis.

4.1) Save data splits

Running this script will create dataset splits in `saved/splits/` as follows (e.g. for BPR on ml-100k):

```
python save_split.py --model BPR --dataset ml-100k \
  --config_files recbole/config/BPR/ml-100k.yaml \
  --output_dir saved/splits
```

Important: because `data_preparation` splits the dataset using the same `seed` and `eval_args` as any real training run of the same config, the TSVs you get here are exactly the train/valid/test partition your model was trained/evaluated on – as long as you point this script at the *same* config file(s) you trained with.

4.2) Save recommendation results

Running this script will create recommendation results in `saved/recommendations/` as follows:

```
python save_recommendations.py \
  --model_file saved/<dataset>-<model_name>-<timestamp>.pth \
  --k 10 \
  --output_dir saved/recommendations
```

Run this once per trained model. Because the JSON records `dataset`, `seed`, and `eval_args`, you can later confirm two recommendation files were produced under matching conditions before comparing them.